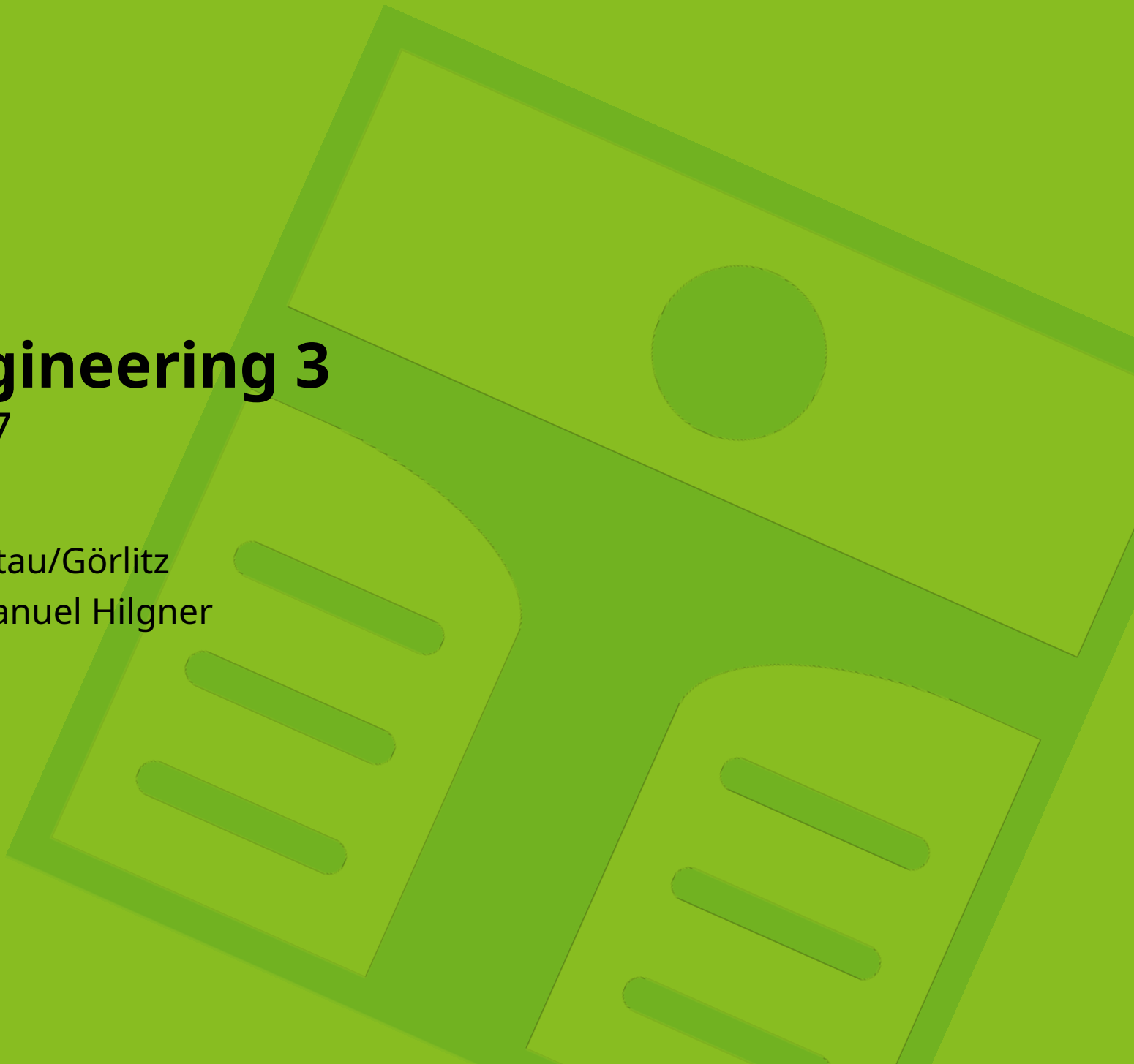
Web Engineering 3

Vorlesung 7

Hochschule Zittau/Görlitz
Christopher-Manuel Hilgner



Agenda

Vorlesung

- Error Handling in Spring
- API Dokumentation

Seminar

- Error Handling
- Build Container für Spring Anwendung
- Container für Spring Anwendung

Error Handling

Custom Template Messages

```kotlin
class ErrorMessageModel(
  var status: Int? = null,
  var message: String? = null
)
```

- `status` speichert den HTTP Status Code
- `message` speichert eine benutzerdefinierte Nachricht, die an das Frontend geschickt wird, wenn ein Error geworfen wird

Custom Template Messages

Kotlin

```kotlin
@ControllerAdvice
class ExceptionControllerAdvice {
  fun handleIllegalStateException(
    ex: IllegalStateException
  ): ResponseEntity<ErrorMessageModel> {
    val errorMessage = ErrorMessageModel(
      HttpStatus.NOT_FOUND.value(),
      ex.message
    )
    return ResponseEntity(
      errorMessage,
      HttpStatus.BAD_REQUEST
    )
  }
}
```

Custom Template Messages

```kotlin
class ArticleNotFoundException(
  message: String
  ): RuntimeException(message) {

}
```

- Es können eigene Exceptions erstellt werden
- Diese Exceptions sollten spezifische Szenarien im Backend abdecken

Custom Template Messages

```kotlin
fun handleArticleNotFoundException(ex:
ArticleNotFoundException):
ResponseEntity<ErrorMessageModel> {
  val errorMessage = ErrorMessageModel(
    HttpStatus.NOT_FOUND.value(),
    ex.message
  )
  return ResponseEntity(
    errorMessage, HttpStatus.NOT_FOUND
  )
}
```

Einbindung

```kotlin
fun getArticle(id: String): ArticleModel {
    return articles.find { articleModel -> articleModel.id == id } ?: throw ArticleNotFoundException("Article not found")
}
```

Einbindung

```kotlin
fun createArticle(title: String): ArticleModel {
    val article = (articles.find { articleModel ->
    articleModel.title == title })
    if (article != null) {
        throw IllegalStateException("Article with the same title
        already exists")
    }
    return ArticleModel("4", title)
}
```

Einbindung

```kotlin
fun updateArticle(id: String, title: String): ArticleModel {
  val article = (articles.find { articleModel ->
    articleModel.id == id
  } ?: throw ArticleNotFoundException("Article not found"))
  if (title.length > 50) {
    throw IllegalArgumentException("Article title too long")
  }
  article.title = title
  return article
}
```

ResponseStatusException Class

Kotlin

```kotlin
@PutMapping
fun updateArticle(
  @RequestParam id: String,
  @RequestParam title: String
): ArticleModel {
  try {
    return articleService.updateArticle(id, title)
  } catch (ex: IllegalArgumentException) {
    throw ResponseStatusException(
      HttpStatus.BAD_REQUEST.localizedMessage, ex
    )
  }
}
```

ResponseStatusException Class

- Gut um Exceptions dynamisch zu handeln
- Exceptions können in der Methoden Definition des Controllers verarbeitet werden
- Es wird kein globaler Controller benötigt
- Es müssen Orte definiert werden, wo spezifische Exceptions gehandelt werden

API Dokumentation

API Dokumentation

SpringDoc

Organisation mit Tags

Controller

```kotlin
@Tag(
  name = "user_controller",
  description = "Controller for CRUD operations on the user"
)
@RestController
@RequestMapping("/users")
class UserController(
  private val userService: UserService
) {
  // Controller Methods
}
```

Organisation mit Tags

Controller

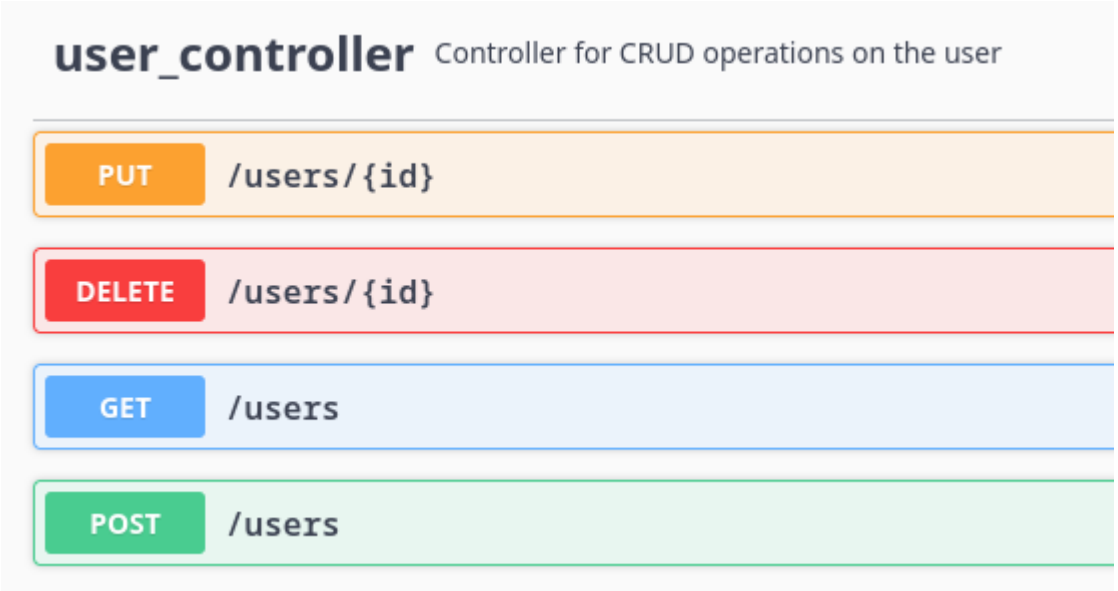


Figure 1: In der Swagger UI wird der User Controller mit den Werten aus `@Tag` angezeigt.

Organisation mit Tags

Methoden

```kotlin
@Tag(
    name = "getUsers",
    description = "Get a List of Users according to provided id and username."
)
@GetMapping
@ResponseStatus(HttpStatus.OK)
fun getUsers(
    @RequestParam id: Long?,
    @RequestParam username: String?
): List<GetUserDto> {
    return userService.getUsers(id, username)
}
```

Organisation mit Tags

Methoden

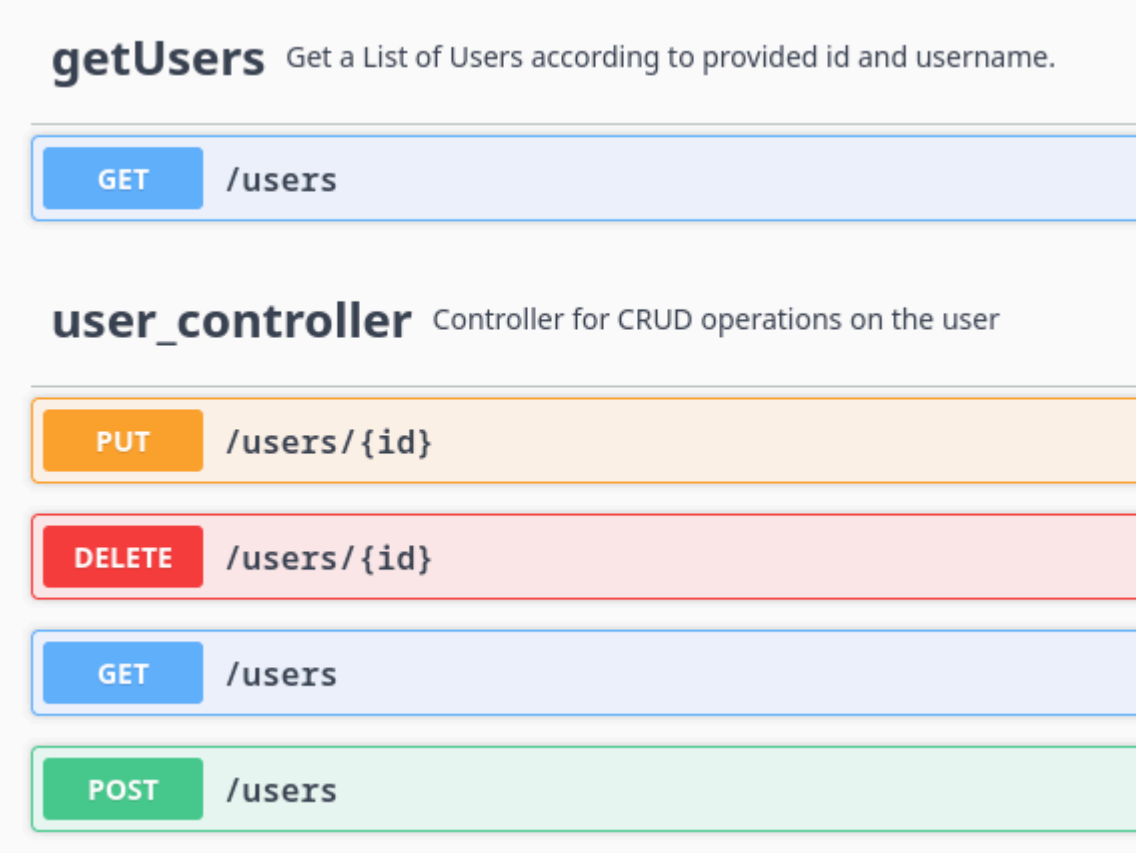


Figure 2: Methode wird mit Name und Beschreiung versehen. Sie wird dabei nicht mehr innerhalb des Controller Tags gerendert.

Organisation mit Tags

Mehrere Tags

```kotlin
@Tag(name = "...", description = "...")
@Tag(name = "GET Methods")
@GetMapping
@ResponseStatus(HttpStatus.OK)
fun getUsers(
  @RequestParam id: Long?,
  @RequestParam username: String?
): List<GetUserDto> {
  return userService.getUsers(id, username)
}
```

Organisation mit Tags

Mehrere Tags

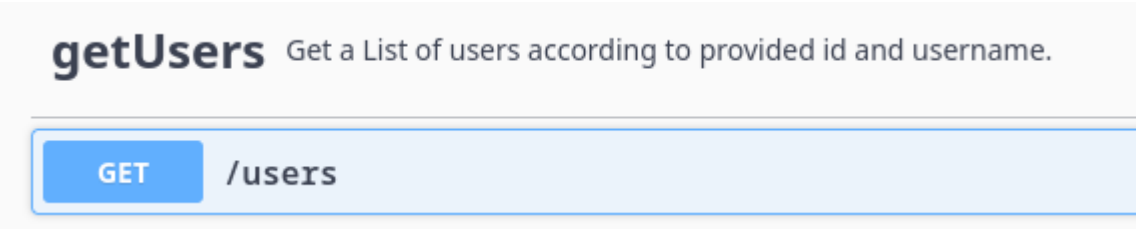


Figure 3: Eintrag in Swagger mit Name und Beschreibung

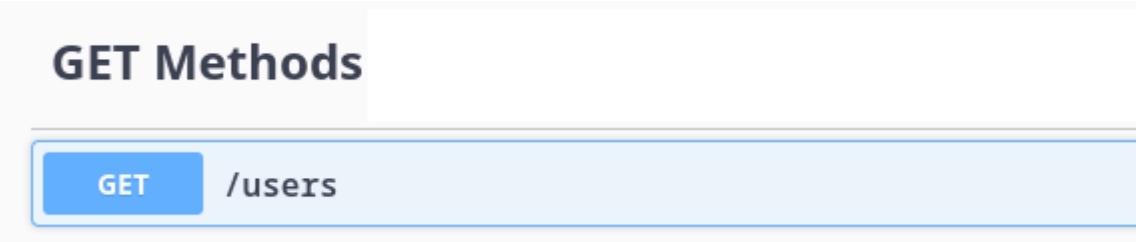


Figure 4: Eintrag in Swagger mit Name

Organisation mit Tags

Mehrere gleiche Tags an Methoden

```kotlin
@Tag(name = "GET Methods")
@GetMapping
@ResponseStatus(HttpStatus.OK)
fun getUsers(): List<GetUserDto> {
  // Method Body
}
```

```kotlin
@Tag(name = "GET Methods")
@GetMapping
@ResponseStatus(HttpStatus.OK)
fun getTodoItems(): List<GetTodoItemDto> {
    // Method Body
}
```

Organisation mit Tags

Mehrere gleiche Tags an Methoden

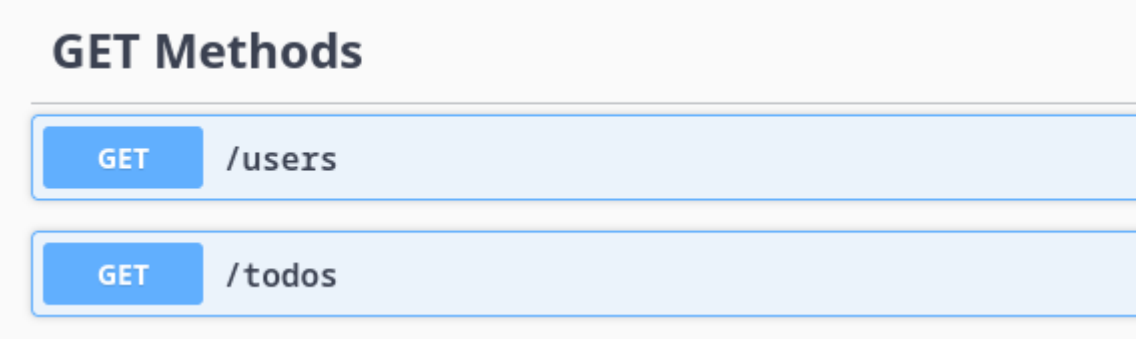


Figure 5: Methoden mit gleichen Tags werden in gemeinsame Gruppen gepackt.

Sortierung der Methoden

- Auflistung anhand der HTTP Methoden:
  1. PUT
  2. POST
  3. GET
  4. DELETE
- `@Tag` fügt ein Feld bei `openAPI.tags` hinzu
- Wenn Felder wie `description` oder `externalDocs` hinzugefügt werden, wird die Sortierung beeinflusst
- Wenn ein `@Tag` auf mehrere Operationen zutrifft, wird wieder auf die HTTP Methoden zurückgegriffen

Sortierung der Methoden

`tagsSorter` Property

application.properties properties

```properties
springdoc.swagger-ui.tagsSorter=alpha
```

Alphabetische Sortierung der Tag Gruppen

application.properties properties

```properties
springdoc.writer-with-order-by-keys=true
```

Sortierung nach Pfaden. Tags werden aber priorisiert.

Sortierung der Methoden

`tags` in `openapi.yaml`

openapi.yml YAML

```yaml
tags:
- name: books
  description: Everything about your books
  externalDocs:
    url: http://docs.my-api.com/books.htm
- name: create
  description: Add a book to the inventory
  externalDocs:
    url: http://docs.my-api.com/add-book.htm
```

Das `books` Tag wird zuerst gerendert. Danach folgt das `create` Tag.

Sortierung der Methoden

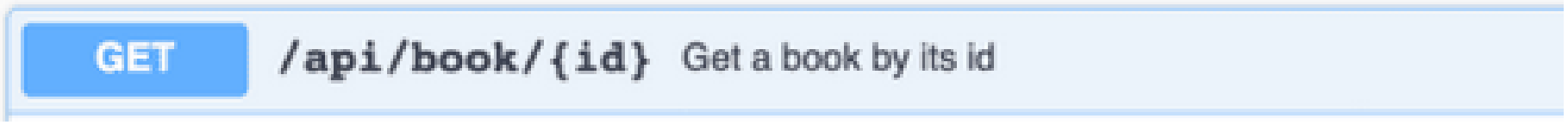
`@OpenAPIDefinition`

Kotlin

```kotlin
@OpenAPIDefinition(
    tags = {
      @Tag(
        name = "create",
        description = "Add book to inventory"
      ),
      @Tag(
        name = "delete",
        description = "Delete book from inventory"
      ),
    }
)
class BooksController_2 { ... }
```

@Operation

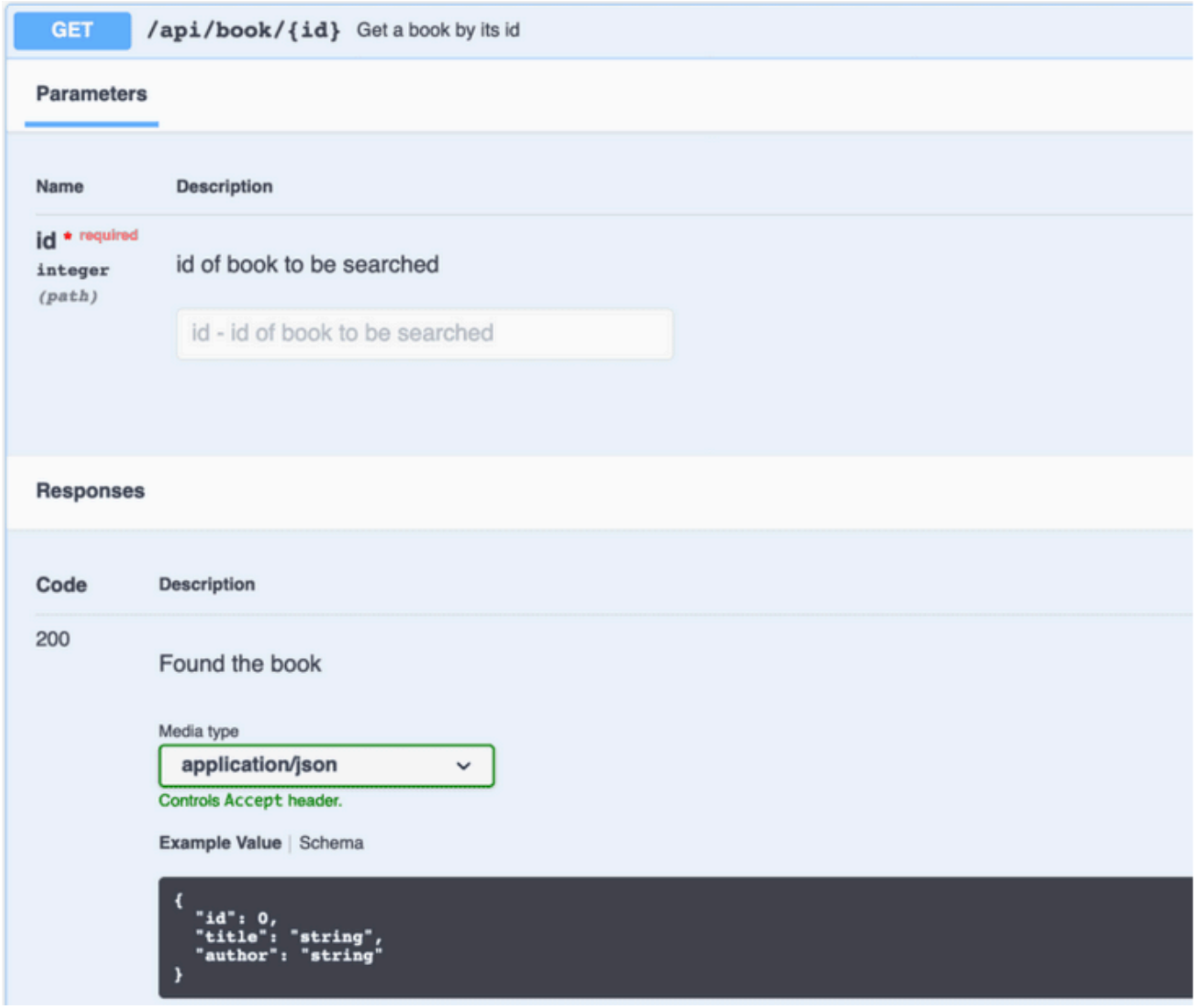
```java
@Operation(
  summary = "Get a book by its id"
)
public Book findById(
  @PathVariable long id
) {
  return repository.findById(id).orElseThrow(() -> new BookNotFoundException());
}
```



@ApiResponses

```java
@ApiResponses(
  value = {
    @ApiResponse(
      responseCode = "200",
      description = "Found the book",
      content = {
        @Content(
          mediaType = "application/json",
          schema = @Schema(implementation = Book.class)
        )
      }
    )
  }
)
public Book findById() {}
```

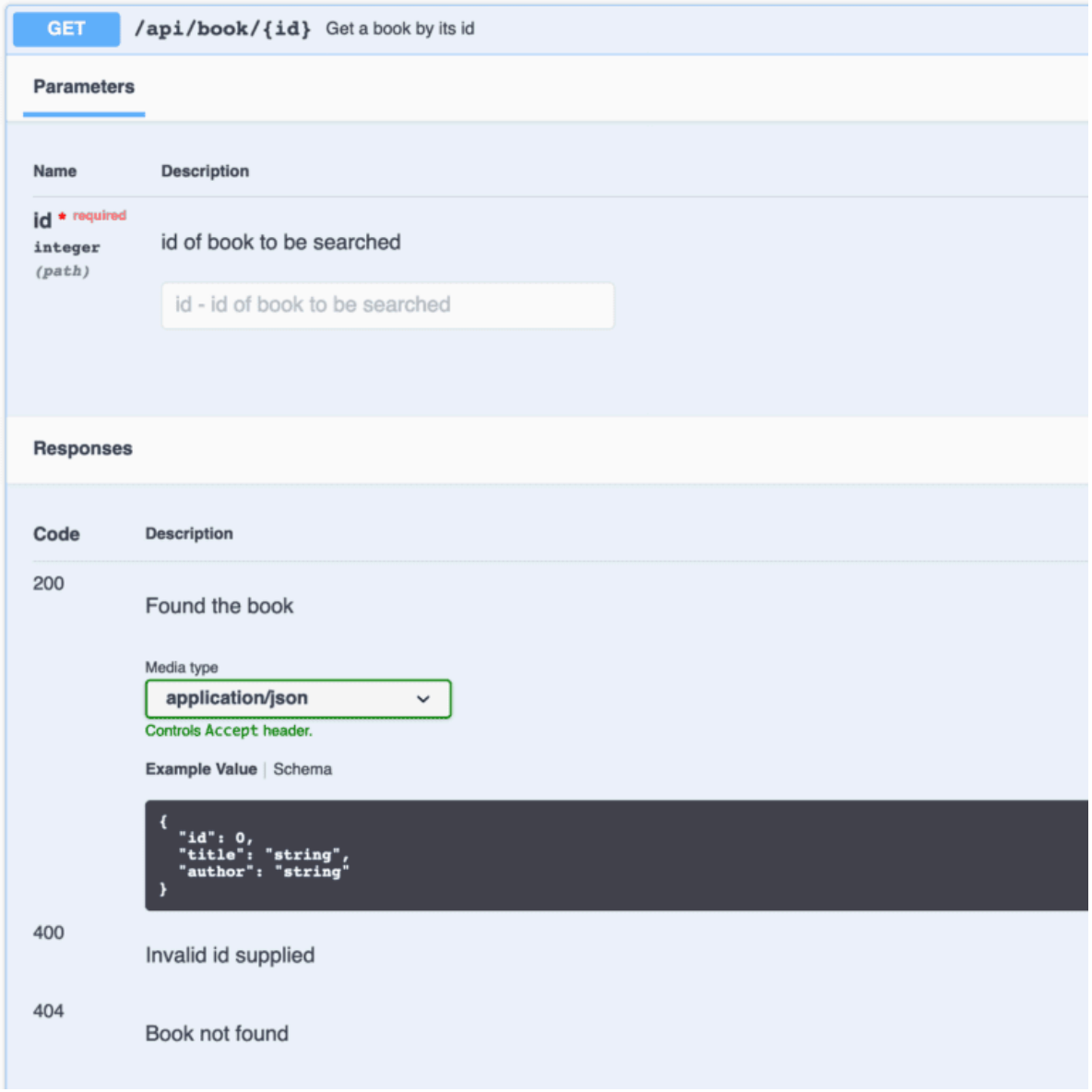
@ApiResponses



@ApiResponses

```java
@ApiResponses(
    value = {
      ...
      @ApiResponse(
        responseCode = "400",
        description = "Invalid id supplied",
        content = @Content
      ),
    }
)
public Book findById() {}
```

@ApiResponses



Alternativen zu Swagger

Rapidoc

Redocly

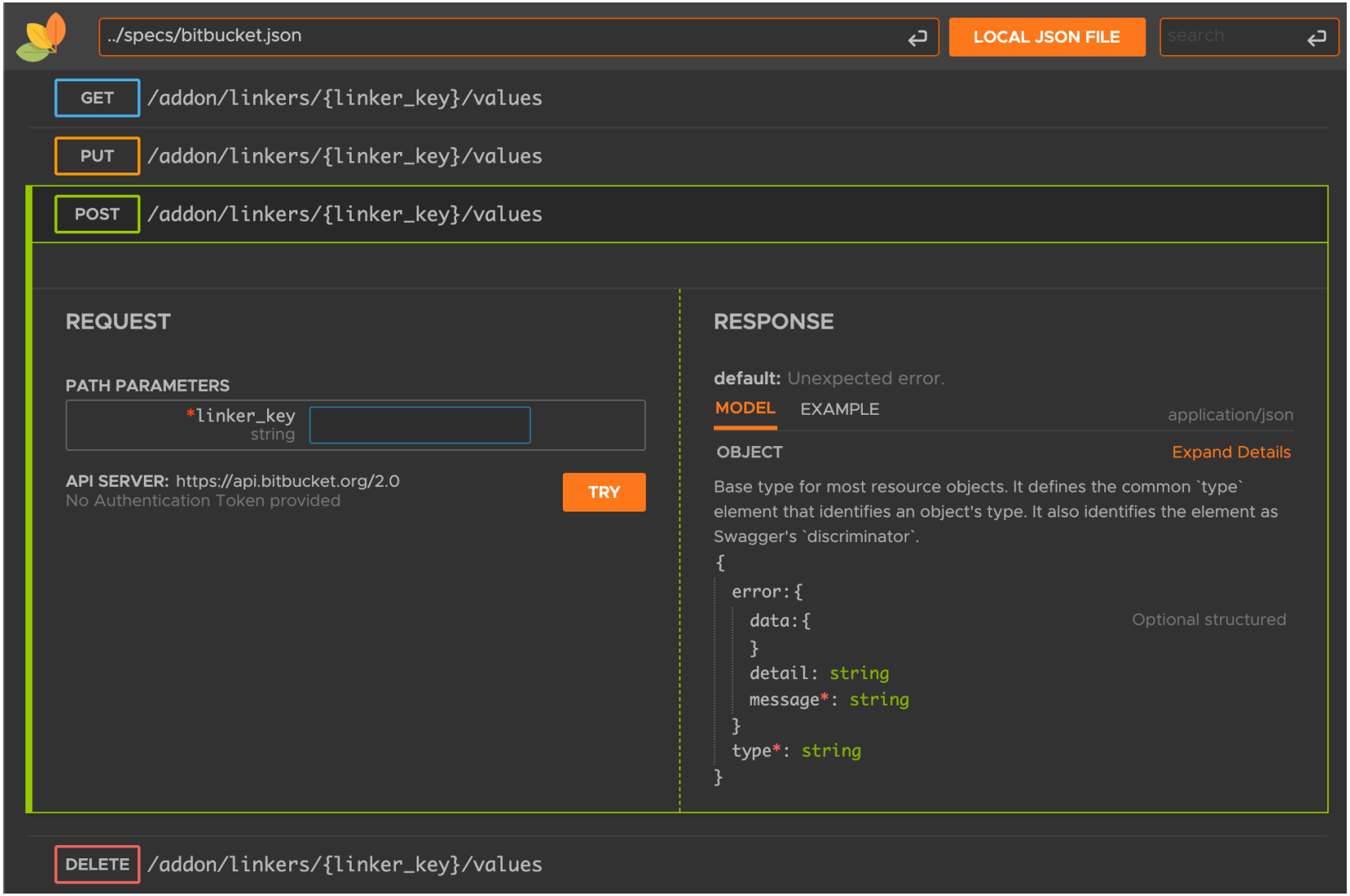
OpenAPI Explorer by *Authress*

...

API Dokumentation

RapiDoc

Rapidoc


../specs/bitbucket.json
LOCAL JSON FILE
search
GET /addon/linkers/{linker_key}/values
PUT /addon/linkers/{linker_key}/values
POST /addon/linkers/{linker_key}/values
REQUEST
PATH PARAMETERS
*linker_key
string
API SERVER: https://api.bitbucket.org/2.0
No Authentication Token provided
TRY
RESPONSE
default: Unexpected error.
MODEL
EXAMPLE
application/json
OBJECT
Expand Details
Base type for most resource objects. It defines the common `type` element that identifies an object's type. It also identifies the element as Swagger's `discriminator`.
{
error:{
data:{
Optional structured
}
detail: string
message*: string
}
type*: string
}
DELETE /addon/linkers/{linker_key}/values

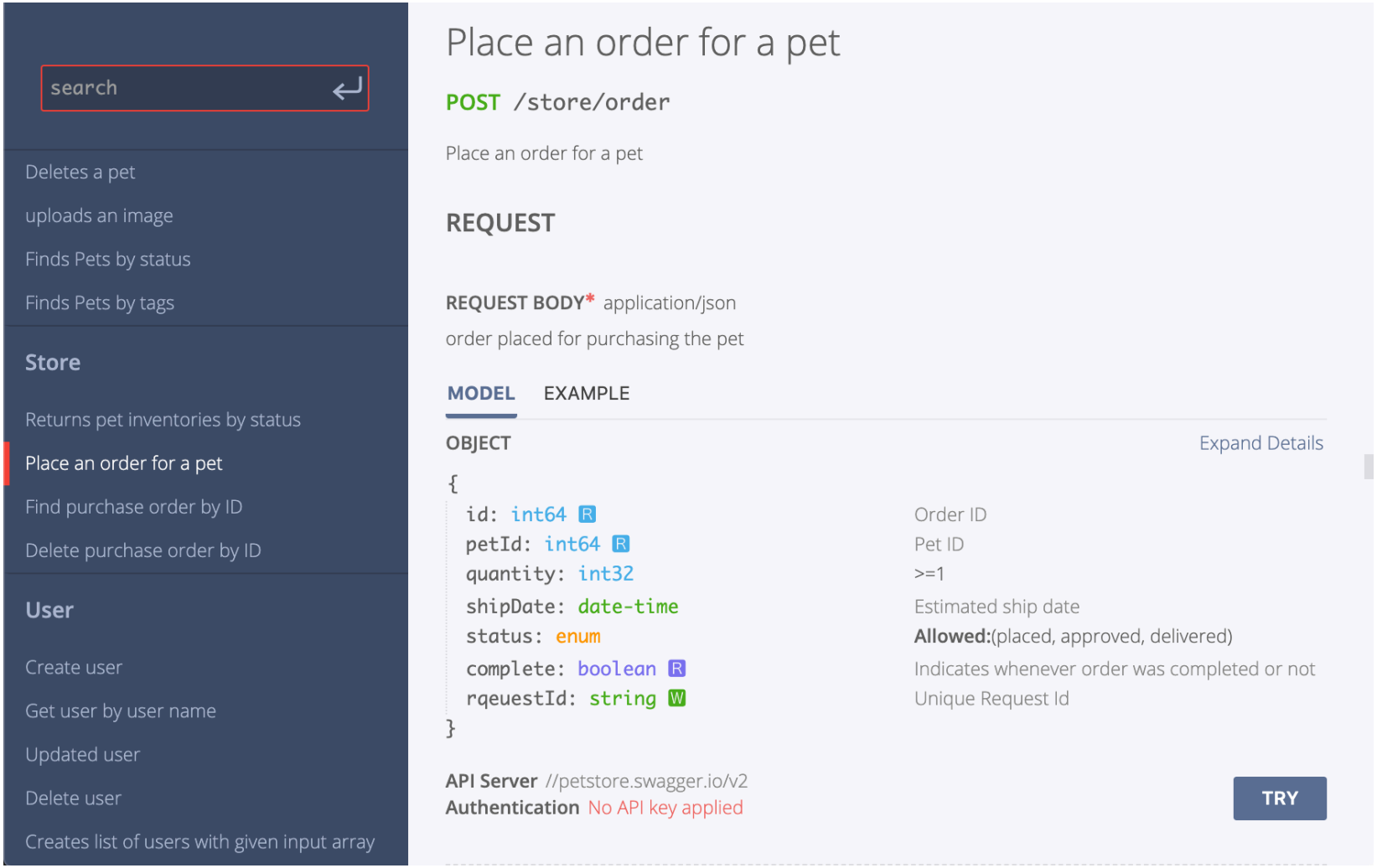
Rapidoc

HTML

```html
<!doctype html>
<html>
  <head>
    <meta charset="utf-8">
    <script type="module" src="https://unpkg.com/rapidoc/dist/
    rapidoc-min.js"></script>
  </head>
  <body>
    <rapi-doc
      spec-url = "https://petstore.swagger.io/v2/swagger.json"
    > </rapi-doc>
  </body>
</html>
```

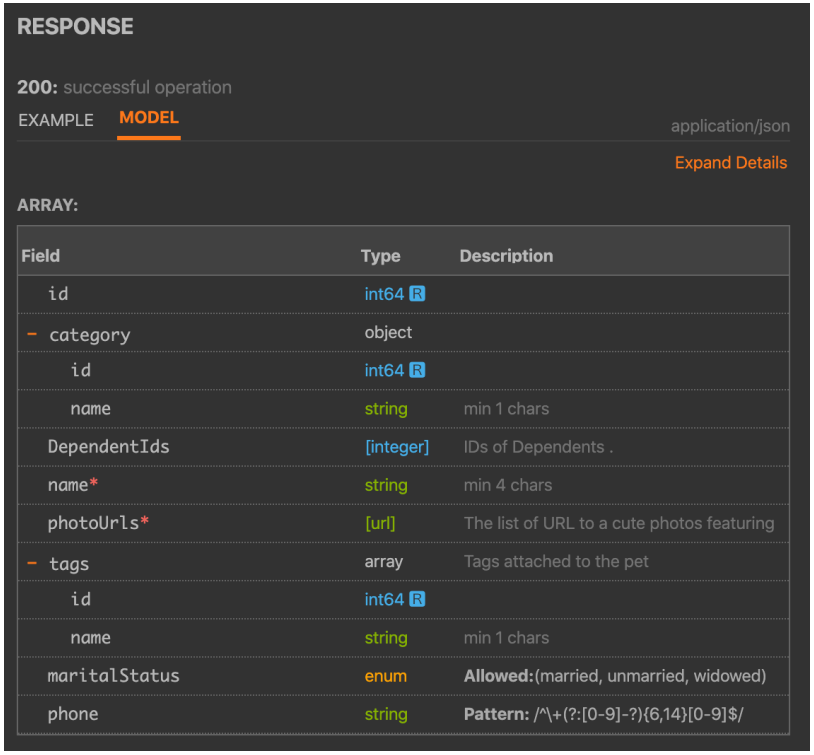
Rapidoc

Wiki Rendering

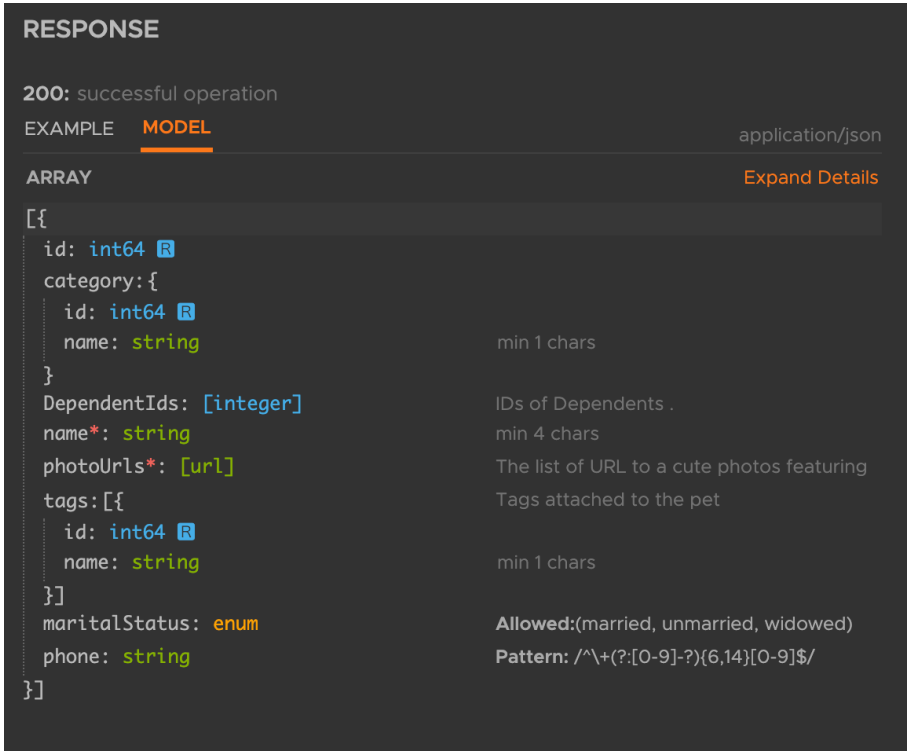


Rapidoc

Objekt Darstellung



Tabellarisch



Baumansicht

MENSA IST VORBEI